

INTRODUCCIÓN A LA CIENCIA DE DATOS

# INFORME TAREA 2: Clasificación de artículos de prensa

Bruno Fabre Hernandez  
Santiago Burgueño Paysse

25 de Junio 2026

Repositorio: <https://github.com/santiago0111/introcd-2026-grupo-10>

## Contents

---

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>Introducción</b>                                                          | <b>2</b>  |
| <b>1 Parte 1: Dataset y representación numérica de texto</b>                 | <b>2</b>  |
| 1.1 Preparación del dataset . . . . .                                        | 2         |
| 1.2 Verificación del balance de clases . . . . .                             | 3         |
| 1.3 Representación Bag of Words . . . . .                                    | 3         |
| 1.4 Representación TF-IDF . . . . .                                          | 4         |
| 1.5 Visualización SVD sobre TF-IDF . . . . .                                 | 4         |
| <b>2 Parte 2: Entrenamiento y Evaluación de Modelos</b>                      | <b>6</b>  |
| 2.1 Multinomial Naive Bayes . . . . .                                        | 6         |
| 2.2 Validación cruzada y búsqueda de hiperparámetros . . . . .               | 7         |
| 2.3 Entrenamiento final con el mejor modelo . . . . .                        | 8         |
| 2.4 Modelo alternativo: Regresión Logística . . . . .                        | 9         |
| 2.5 Cambio de medio de prensa . . . . .                                      | 10        |
| 2.6 Técnica alternativa de extracción de features: Word Embeddings . . . . . | 11        |
| 2.7 Clasificación a nivel de título . . . . .                                | 11        |
| <b>Conclusiones</b>                                                          | <b>12</b> |

## Introducción

---

Esta tarea es la continuación de la Tarea 1. Se utilizan los mismos datos del dataset *All the News 2.1* y la función `clean_text()` desarrollada previamente. El objetivo es aplicar técnicas de representación numérica de texto y entrenar modelos de clasificación para predecir a qué medio de prensa pertenece cada artículo. El código completo está disponible en el repositorio: <https://github.com/santiago0111/introcd-2026-grupo-10>.

## Parte 1: Dataset y representación numérica de texto

---

### 1.1. Preparación del dataset

Se trabajó con los tres medios de prensa con mayor cantidad de artículos del dataset:

| Medio              | Artículos totales |
|--------------------|-------------------|
| Reuters            | 9.431             |
| The New York Times | 2.840             |
| CNBC               | 2.623             |

Table 1: Medios seleccionados y cantidad de artículos.

Se aplicó la función `clean_text()` de la Tarea 1 sobre la combinación de título y cuerpo del artículo. Esta función elimina el dateline, convierte a minúsculas, elimina URLs y puntuación, y colapsa espacios múltiples.

El dataset resultante (14.894 artículos) se particionó en **70% train** y **30% test** con muestreo estratificado (`stratify=y`, `random_state=42`), garantizando que la proporción de cada medio se mantenga igual en ambos conjuntos:

| Partición | Artículos |
|-----------|-----------|
| Train     | 10.425    |
| Test      | 4.469     |

Table 2: Tamaño de las particiones train y test.

## 1.2. Verificación del balance de clases

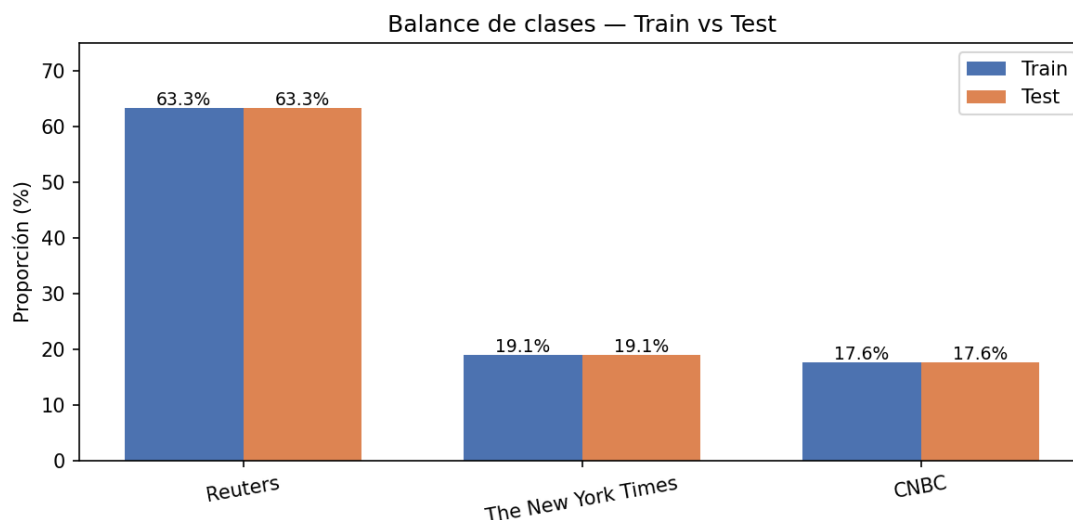


Figure 1: Balance de clases en los conjuntos de train y test.

El gráfico confirma que la estratificación funcionó correctamente: las proporciones de cada medio son prácticamente idénticas en train y test. Reuters representa aproximadamente el 63% del dataset, mientras que NYT y CNBC representan alrededor del 19% y 18% respectivamente. Este desbalance tendrá implicancias importantes en el comportamiento de los modelos, como se analizará en la Parte 2.

## 1.3. Representación Bag of Words

Bag of Words (BoW) representa cada artículo como un vector de conteos sobre el vocabulario construido a partir del conjunto de entrenamiento, produciendo una **matriz documento-término** de dimensión artículos  $\times$  términos. La representación es inherentemente *sparse* — cada artículo activa solo una fracción pequeña del vocabulario total — y descarta el orden de las palabras: el modelo captura presencia y frecuencia, pero no estructura sintáctica ni contexto.

El vectorizador se ajusta (**fit**) únicamente sobre el conjunto de entrenamiento y se aplica (**transform**) sobre el test, evitando filtración de información entre ambas particiones.

| Dimensión                          | Valor         |
|------------------------------------|---------------|
| Artículos (filas)                  | 10.425        |
| Palabras en vocabulario (columnas) | 86.698        |
| Celdas totales                     | ~904 millones |
| Celdas no-nulas                    | 0,247%        |

Table 3: Dimensiones de la matriz BoW del conjunto de entrenamiento.

Almacenar la matriz completa en formato denso requeriría varios GB de RAM. Una representación *sparse* guarda únicamente los valores no-nulos junto con sus coordenadas, reduciendo el uso de memoria en más de 400 veces.

## 1.4. Representación TF-IDF

Un **n-grama** es una secuencia de  $N$  palabras consecutivas. Con `ngram_range=(1,2)` el vectorizador considera tanto palabras individuales (unigramas) como pares de palabras adyacentes (bigramas). Por ejemplo, “nueva york” como bigrama captura información que se perdería tratando “nueva” y “york” como términos independientes.

TF-IDF mejora BoW penalizando palabras muy frecuentes en todos los documentos y premiando palabras específicas de ciertos artículos. El **TF** (*Term Frequency*) mide la frecuencia de la palabra en el documento actual; el **IDF** (*Inverse Document Frequency*) penaliza palabras que aparecen en muchos documentos — palabras como “the” o “said” tienen IDF cercano a 0. El peso final  $\text{TF-IDF} = \text{TF} \times \text{IDF}$  resulta alto para palabras frecuentes en un artículo pero raras en el corpus.

Se utilizó la configuración `stop_words="english"`, `use_idf=True`, que adicionalmente elimina las palabras más comunes del inglés antes de la vectorización. El vocabulario TF-IDF resultante tiene **86.393 términos**.

## 1.5. Visualización SVD sobre TF-IDF

Tras aplicar TF-IDF, cada artículo queda representado como un vector de 86.393 dimensiones — un espacio imposible de visualizar directamente. Para explorar si los artículos de los tres medios forman grupos distinguibles, se aplicó **Singular Value Decomposition** (SVD), una técnica de álgebra lineal que reduce la dimensionalidad del espacio preservando la mayor cantidad posible de su estructura.

SVD opera proyectando todos los artículos sobre las direcciones del espacio donde los datos presentan mayor dispersión entre sí. La primera componente (SVD1) captura la dirección de máxima varianza; la segunda (SVD2) captura la segunda dirección más informativa, con la restricción de ser perpendicular a SVD1 para no redundar en la misma información. Al retener solo 2 componentes, cada artículo queda representado por un par de coordenadas que puede graficarse.

Estas coordenadas son combinaciones lineales de miles de términos y no tienen una interpretación semántica directa. Lo que sí es interpretable es la posición relativa entre artículos: puntos cercanos en el gráfico comparten un perfil de vocabulario similar.

Se utilizó **TruncatedSVD** en lugar de PCA clásico porque opera directamente sobre matrices *sparse*, evitando la conversión a formato denso que PCA requiere y que produciría un error de memoria con matrices de estas dimensiones.

## Comparación de configuraciones

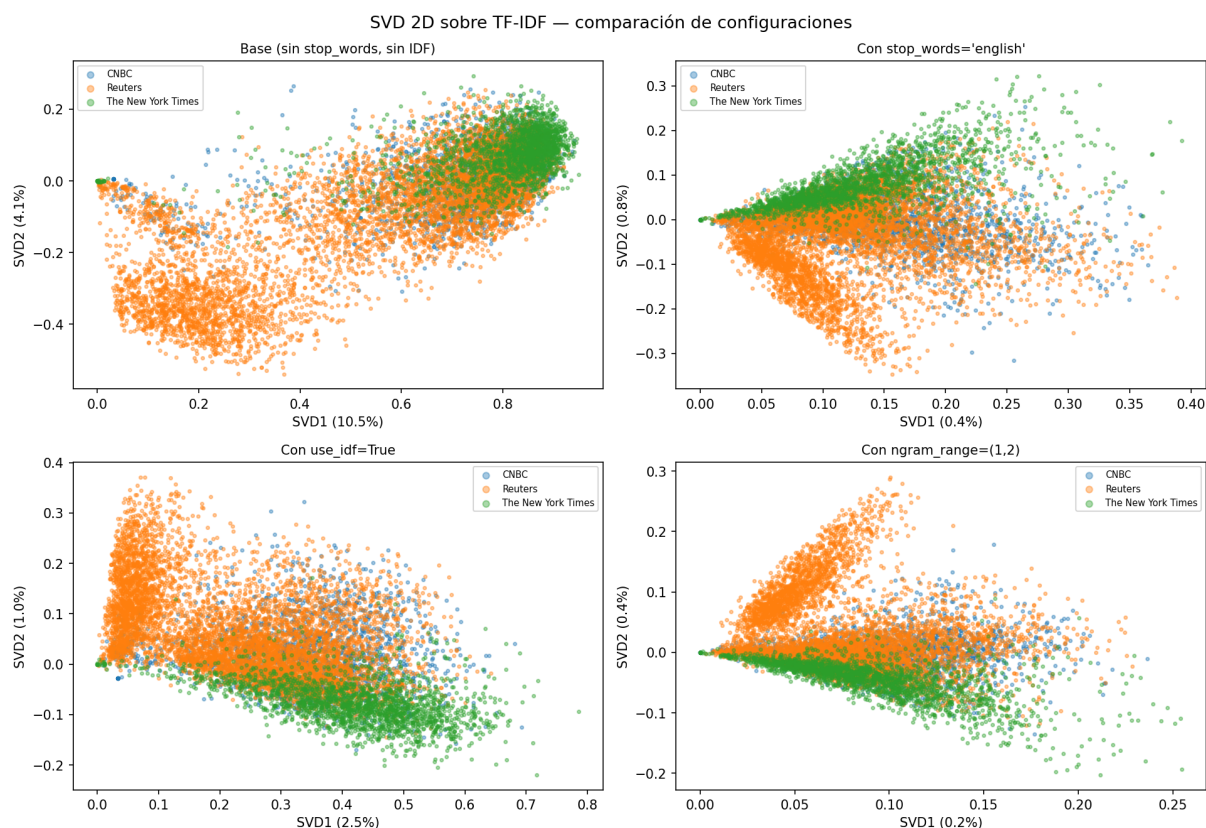


Figure 2: Visualización SVD (2 componentes) con cuatro configuraciones de vectorización TF-IDF.

Los cuatro paneles corresponden a las siguientes configuraciones:

- **Base:** `TfidfVectorizer(use_idf=False)` — stop words no eliminadas, sin IDF. SVD1 = 10,5%.
- **Con stop\_words:** `TfidfVectorizer(stop_words="english", use_idf=True)` — stop words eliminadas + IDF. SVD1 = 0,4%.
- **Con use\_idf=True:** `TfidfVectorizer(use_idf=True)` — solo IDF, stop words no eliminadas. SVD1 = 2,5%.
- **Con ngram\_range=(1,2):** `TfidfVectorizer(stop_words="english", use_idf=True, ngram_range=(1,2))` — stop words eliminadas + IDF + bigramas. SVD1 = 0,2%.

En todos los gráficos se observa superposición considerable entre los tres medios. Esto no implica que el problema de clasificación sea difícil, sino que **2 componentes son insuficientes** para capturar la separación real que existe en las 86.393 dimensiones originales.

En el caso base, SVD1 captura el 10,5% de la varianza, dominada por palabras muy frecuentes como “the”, “said”, “a” que no fueron eliminadas — producen mucha varianza pero poca señal discriminativa. Al eliminar stop words y aplicar IDF esa varianza se redistribuye hacia términos más informativos y SVD1 cae a 0,4%. Con bigramas el vocabulario se expande aún más y la varianza se diluye entre más dimensiones, llevando SVD1 a 0,2%.

## Varianza explicada

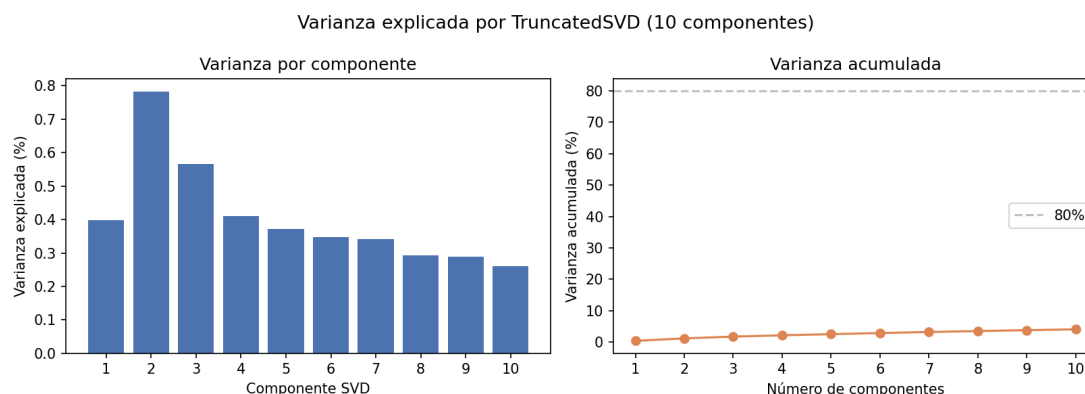


Figure 3: Varianza explicada acumulada por TruncatedSVD con hasta 10 componentes.

Con 10 componentes, TruncatedSVD explica apenas el **4,06% de la varianza total**. Esto indica que la información discriminativa no está concentrada en pocas dimensiones sino distribuida de forma muy homogénea. La superposición visual entre clases en los gráficos 2D no refleja la dificultad real del problema: los modelos de la Parte 2 logran separar los medios con un rendimiento significativamente mejor del que los gráficos sugerirían.

La separación observada puede deberse a diferencias temáticas (cada medio cubre ciertos temas con mayor frecuencia), diferencias de estilo editorial (registro, longitud, adjetivos), o pistas explícitas como menciones directas al nombre del medio. La eliminación del dateline en `clean_text()` mitiga este último factor parcialmente.

## Parte 2: Entrenamiento y Evaluación de Modelos

### 2.1. Multinomial Naive Bayes

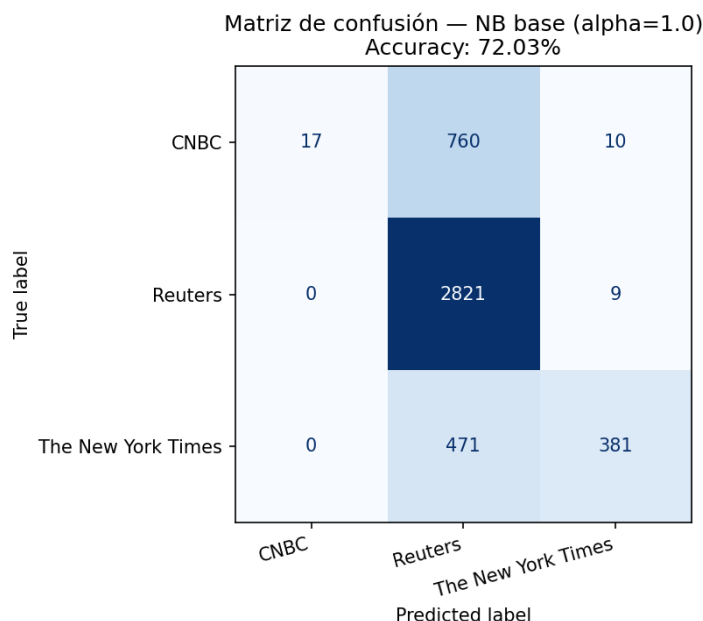
Naive Bayes aplica el teorema de Bayes: dado un artículo con ciertas palabras, calcula la probabilidad de que pertenezca a cada clase y elige la de mayor probabilidad. Se llama “naive” porque asume independencia condicional entre palabras — cada término contribuye de forma independiente a la probabilidad de la clase.

El parámetro `alpha` controla el suavizado de Laplace: evita que palabras no vistas en el entrenamiento colapsen la probabilidad de una clase a cero.

El **accuracy** mide la proporción de artículos clasificados correctamente sobre el total del test:  $\text{Accuracy} = \text{predicciones correctas} / \text{total de artículos}$ . Con desbalance de clases este número puede ser engañoso — un clasificador trivial que prediga siempre Reuters obtendría 63% de accuracy sin aprender nada.

**Resultados con `alpha=1.0` (valor por defecto):**

| Medio              | Precision     | Recall | F1   | Support     |
|--------------------|---------------|--------|------|-------------|
| CNBC               | 1,00          | 0,02   | 0,04 | 787         |
| Reuters            | 0,70          | 1,00   | 0,82 | 2830        |
| The New York Times | 0,95          | 0,45   | 0,61 | 852         |
| <b>Accuracy</b>    | <b>0,7203</b> |        |      | <b>4469</b> |

Table 4: Métricas por clase — Naive Bayes con  $\alpha=1.0$ .Figure 4: Matriz de confusión — Naive Bayes con  $\alpha=1.0$ .

Los resultados revelan un modelo fuertemente sesgado hacia la clase mayoritaria. Reuters alcanza  $\text{recall} = 1,00$  pero  $\text{precision} = 0,70$ : el modelo clasifica como Reuters gran parte de los artículos ante cualquier ambigüedad. En el extremo opuesto, CNBC es prácticamente ignorada —  $\text{recall} = 0,02$  y  $\text{F1} = 0,04$  — con el 98% de sus artículos absorbidos por Reuters. NYT queda en una posición intermedia con  $\text{recall} = 0,45$ .

El accuracy global del 72% resulta engañoso dado el desbalance de clases. El F1 macro de 0,49 refleja con más fidelidad la capacidad real del modelo.

## 2.2. Validación cruzada y búsqueda de hiperparámetros

Los resultados del modelo base muestran que el valor por defecto de **alpha** no es necesariamente el óptimo. Para encontrar el mejor hiperparámetro es necesario evaluar distintos valores — pero esa evaluación no puede hacerse sobre el test, ya que el conjunto de test debe permanecer completamente aislado hasta el final. En su lugar, la selección de hiperparámetros se realiza sobre el **conjunto de validación**, construido mediante **cross-validation de 5 folds** dentro del train:

```
Dataset completo
+-- TRAIN (70%) <- cross-validation trabaja aca para elegir alpha
```



```

+-- TEST (30%) <- intacto, solo se usa al final una vez

```

**GridSearchCV** automatiza este proceso para cada valor de **alpha**, entrenando en total  $7 \times 5 = 35$  modelos:

| Alpha      | Accuracy promedio (CV) | Desv. estándar |
|------------|------------------------|----------------|
| 0,01       | 0,8046                 | 0,0042         |
| <b>0,1</b> | <b>0,8110</b>          | <b>0,0035</b>  |
| 0,5        | 0,7464                 | 0,0061         |
| 1,0        | 0,7122                 | 0,0044         |
| 2,0        | 0,6700                 | 0,0025         |
| 5,0        | 0,6359                 | 0,0009         |
| 10,0       | 0,6332                 | 0,0002         |

Table 5: Resultados de GridSearchCV por valor de **alpha**. Mejor configuración en negrita.

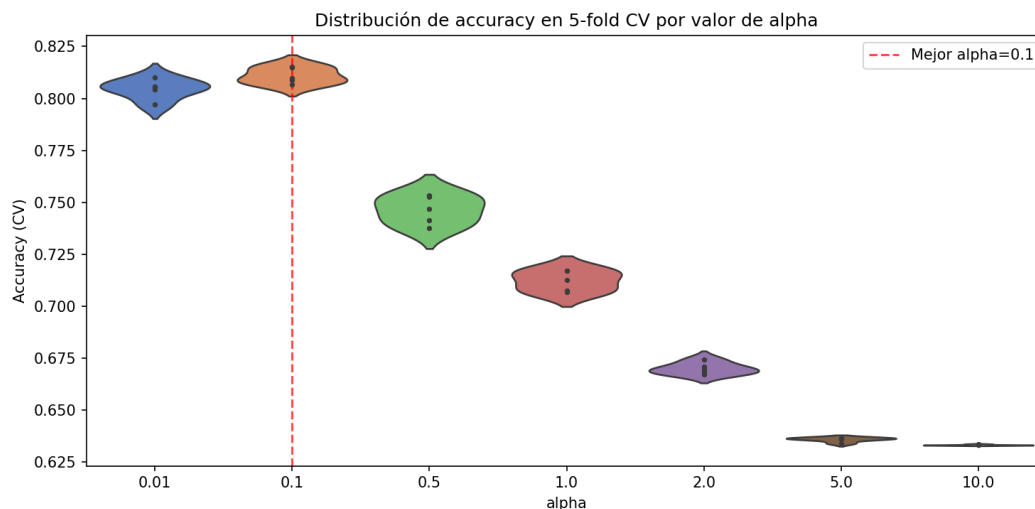


Figure 5: Distribución del accuracy en los 5 folds para cada valor de **alpha** (violin plot).

Alpha= 0,1 tiene el mejor promedio con variabilidad razonable. Valores altos de **alpha** producen modelos demasiado simples que no capturan bien las diferencias entre clases.

### 2.3. Entrenamiento final con el mejor modelo

Con **alpha=0.1**, se reentrenó el modelo sobre **todo el conjunto de entrenamiento** y se evaluó por primera vez sobre el test:

| Medio              | Precision     | Recall | F1   | Support     |
|--------------------|---------------|--------|------|-------------|
| CNBC               | 0,84          | 0,33   | 0,47 | 787         |
| Reuters            | 0,82          | 0,94   | 0,88 | 2830        |
| The New York Times | 0,76          | 0,81   | 0,78 | 852         |
| <b>Accuracy</b>    | <b>0,8085</b> |        |      | <b>4469</b> |

Table 6: Métricas por clase — Naive Bayes con  $\alpha=0.1$  (modelo final).

La mejora es significativa: de 72,03% a **80,85%** de accuracy, y el F1 macro pasó de 0,49 a 0,71. El recall de CNBC mejoró de 0,02 a 0,33.

Los modelos basados en BoW/TF-IDF tienen limitaciones estructurales: el orden de las palabras se pierde, sinónimos tienen columnas separadas sin que el modelo sepa que son similares, y palabras no vistas en el train son ignoradas en el test.

## 2.4. Modelo alternativo: Regresión Logística

La Regresión Logística aprende un peso por cada término del vocabulario. Para clasificar un artículo, calcula un score como suma ponderada de sus valores TF-IDF y aplica la función *softmax* para convertirlo en probabilidades. A diferencia de Naive Bayes, no asume independencia entre *features* — puede capturar correlaciones entre palabras.

| Medio              | Precision     | Recall | F1   | Support     |
|--------------------|---------------|--------|------|-------------|
| CNBC               | 0,88          | 0,51   | 0,65 | 787         |
| Reuters            | 0,85          | 0,98   | 0,91 | 2830        |
| The New York Times | 0,94          | 0,83   | 0,88 | 852         |
| <b>Accuracy</b>    | <b>0,8711</b> |        |      | <b>4469</b> |

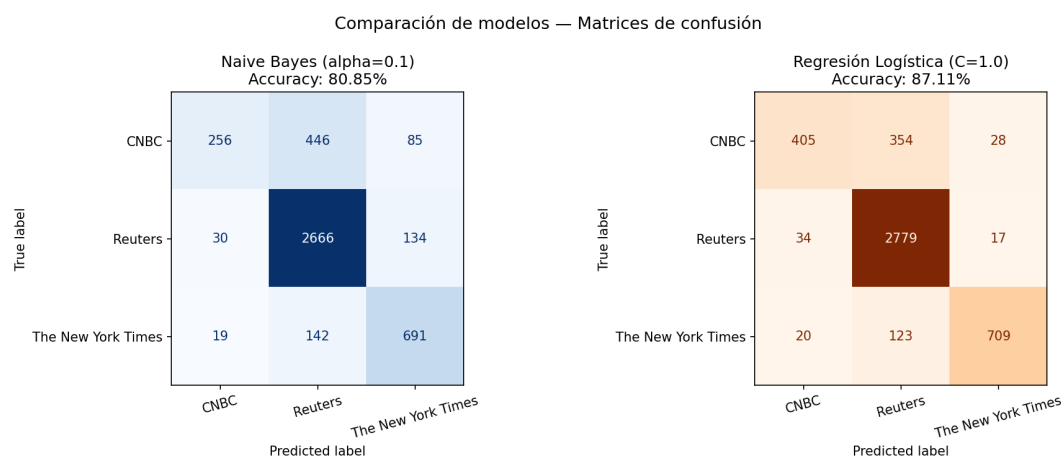
Table 7: Métricas por clase — Regresión Logística ( $C=1.0$ ,  $\text{max\_iter}=1000$ ).

Figure 6: Comparación de matrices de confusión: Naive Bayes (izq.) vs. Regresión Logística (der.).

La Regresión Logística supera al mejor Naive Bayes en todas las métricas: accuracy 87,11% vs. 80,85%, F1 macro 0,82 vs. 0,71. La mejora es especialmente notable en CNBC (recall 0,51 vs. 0,33).

## 2.5. Cambio de medio de prensa

Se evaluó el problema con **Reuters**, **People** y **Business Insider**, elegidos para introducir un desbalance más pronunciado:

| Medio            | Artículos |
|------------------|-----------|
| Reuters          | 9.431     |
| People           | 1.528     |
| Business Insider | 660       |

Table 8: Medios alternativos y cantidad de artículos.

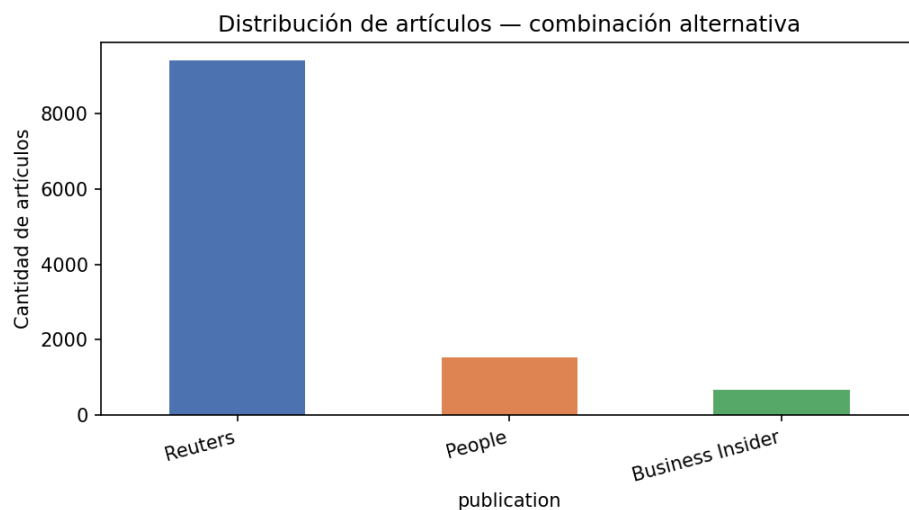


Figure 7: Balance de clases — combinación de medios alternativos.

| Medio            | Precision     | Recall | F1   | Support     |
|------------------|---------------|--------|------|-------------|
| Business Insider | 0,97          | 0,31   | 0,47 | 198         |
| People           | 0,85          | 0,93   | 0,89 | 458         |
| Reuters          | 0,96          | 0,99   | 0,97 | 2830        |
| <b>Accuracy</b>  | <b>0,9406</b> |        |      | <b>3486</b> |

Table 9: Métricas por clase — medios alternativos, Naive Bayes  $\alpha=0.1$ .

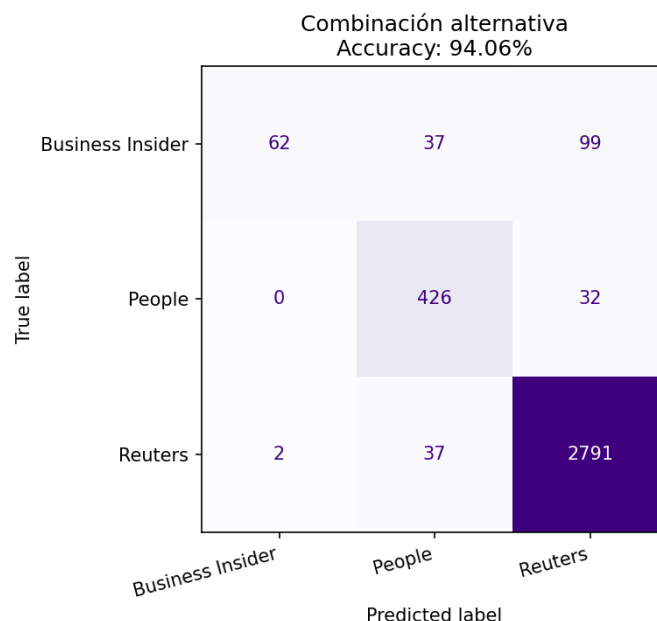


Figure 8: Matriz de confusión — medios alternativos.

El accuracy del 94% es nuevamente engañoso dado que Reuters domina el dataset (81% del test). Business Insider tiene recall de 0,31. People muestra resultados sorprendentemente buenos (recall= 0,93), probablemente porque su vocabulario de entretenimiento es muy distinto al de Reuters.

Para mitigar el desbalance existen técnicas como *oversampling* (SMOTE), *undersampling*, o usar `class_weight='balanced'` en scikit-learn.

## 2.6. Técnica alternativa de extracción de features: Word Embeddings

Los **word embeddings** (Word2Vec, GloVe, FastText, BERT) representan cada palabra como un vector denso de baja dimensión (típicamente 100–300 dimensiones), aprendido de grandes corpus de texto. A diferencia de TF-IDF, los embeddings capturan similitud semántica: palabras con significados parecidos tienen vectores cercanos en el espacio. Modelos más avanzados como BERT además producen un vector distinto para la misma palabra según el contexto en que aparezca.

Para clasificar un documento, se promedian los embeddings de todas sus palabras para obtener un vector representativo. Para tareas donde las diferencias entre clases son principalmente léxicas, TF-IDF puede competir con embeddings simples; la ventaja de los embeddings se vuelve más clara cuando la semántica y el contexto importan más que el vocabulario exacto.

## 2.7. Clasificación a nivel de título

Se repitió el pipeline completo utilizando únicamente el **título del artículo**:

| Configuración                         | Accuracy |
|---------------------------------------|----------|
| Artículo completo (NB, $\alpha=0.1$ ) | 80,85%   |
| Solo título (NB, $\alpha=0.1$ )       | 72,59%   |

Table 10: Comparación de accuracy: artículo completo vs. solo título.

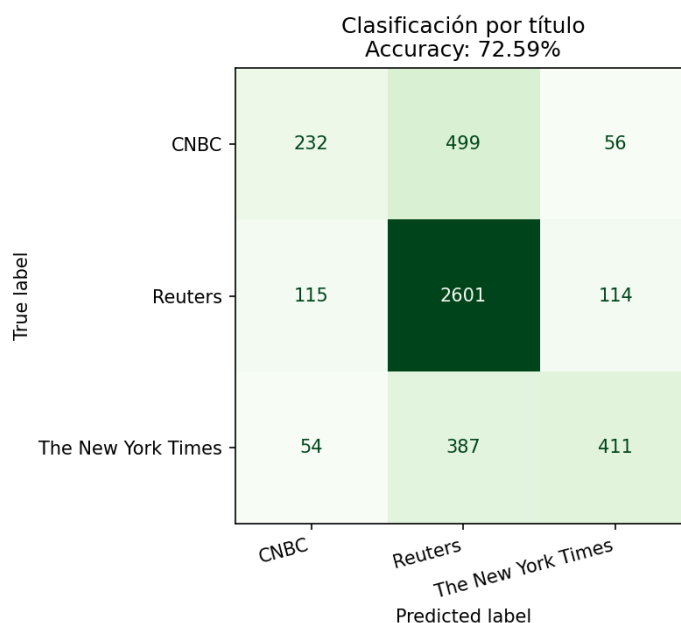


Figure 9: Matriz de confusión — clasificación usando solo el título del artículo.

La caída de  $\sim 8$  puntos de accuracy es esperable dado que el título contiene mucha menos información. Aun así, un 72,6% de accuracy clasificando solo por título indica que los medios tienen estilos de titular diferenciados.

## Conclusiones

La representación TF-IDF con eliminación de stop words es una base sólida para clasificación de texto, alcanzando un 80,85% de accuracy con Naive Bayes optimizado. La Regresión Logística supera claramente a Naive Bayes (87,11% vs. 80,85%), sugiriendo que las correlaciones entre términos son informativas para esta tarea.

El desbalance de clases es el principal desafío del problema: Reuters domina el dataset y los modelos tienden a clasificar hacia esa clase. El accuracy solo no es una métrica suficiente — el F1 por clase revela información que el accuracy oculta.

Con solo 2 componentes SVD se explica menos del 2% de la varianza del espacio TF-IDF, lo que confirma que la separabilidad entre medios existe pero requiere muchas más dimensiones de las que se pueden visualizar. Los modelos basados en BoW/TF-IDF tienen limitaciones estructurales que técnicas como word embeddings o modelos de lenguaje (BERT) superan, a costa de mayor complejidad computacional.